XIAMEN UNIVERSITY MALAYSIA

| | | |
|---|---|---|
| Course Code | : | SOF106 |
| Course Name | : | Principles of Artificial Intelligence |
| Lecturer | : | Dr Hakim Abdulrab |
| Academic Session | : | 2025/04 |
| Assessment Title | : | Final Assessment ( Group Project) |
| Submission Due Date | : | 22nd June 2025 |

Prepared by :

| Student ID | Student Name |
|---|---|
| CYS2309273 | Itanza Joan |
| CYS2309270 | Christian Kenny |
| CYS2309271 | Dillon Nicholas |
| CYS2309274 | Mok Isabella |
| CYS2309276 | Stefanie |
| CYS2309277 | Willim Raymond Russel |

Date Received :

Feedback from Lecturer:

Mark:

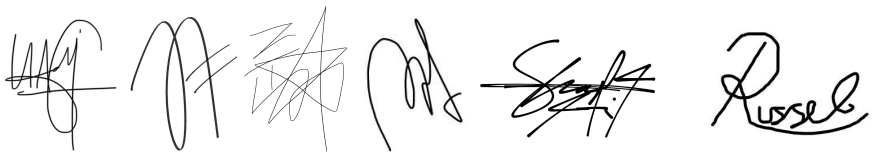
# Own Work Declaration

I/We hereby understand my/our work would be checked for plagiarism or other misconduct, and the softcopy would be saved for future comparison(s).

I/We hereby confirm that all the references or sources of citations have been correctly listed or presented and I/we clearly understand the serious consequence caused by any intentional or unintentional misconduct.

This work is not made on any work of other students (past or present), and it has not been submitted to any other courses or institutions before.

Signature:



Itanza Joan,Christian Kenny, Dillon Nicholas, Mok Isabella, Stefanie, Willim Raymond Russel

Date: 22 June 2025

# Table of Contents

**Abstract** ..... **2**
**Introduction** ..... **2**
**Methodology** ..... **3**
Team Structure and Roles ..... 3
Identifying the Problem ..... 4
Objective ..... 4
Selecting the Approach ..... 5
Literature Review ..... 5
Technology Selection ..... 6
Processing the Data ..... 7
1. Text Normalization ..... 7
2. Tokenization ..... 7
3. Stopword Removal ..... 8
4. Lemmatization or stemming ..... 8
5. Merging Relevant Fields ..... 8
6. Label Encoding ..... 8
**Implementation** ..... **9**
User Interface ..... 9
Text Model ..... 10
TF-IDF Vectorizer ..... 10
Multinomial Naive Bayes ..... 11
OCR Module ..... 12
Integration ..... 14
Evaluation ..... 15
**Validation and Verification** ..... **17**
**Result and Analysis** ..... **18**
Data Collection and Preprocessing ..... 18
Model Training and Evaluation ..... 19
Observations and Limitations ..... 19
**Conclusion** ..... **20**
**References** ..... **21**
**Appendices** ..... **22**
Appendix A: Team Members Task Distribution ..... 22
Appendix B: Directory and Files ..... 24
Appendix C: Graphic User Interface Design ..... 25
Appendix D: Model Evaluation Metrics ..... 26

# AI-Powered Phishing Detection System

Combining Machine Learning and OCR for Enhanced Security

## Abstract

This report presents an AI-powered phishing detection system designed to help users identify phishing messages from texts or screenshots. The system integrates machine learning algorithms, specifically a Naive Bayes classifier, with optical character recognition (OCR) technology and an intuitive graphical user interface to analyze messages and determine their authenticity. It provides a phishing verdict, confidence score, and a warning message to users. In the backend, the system evaluates the model's accuracy, precision, recall, and F1- score to support continuous enhancements by developers. This highly adaptable AI-Powered system significantly enhances cybersecurity by providing a reliable and user-friendly tool for identifying phishing threats. The following sections detail the background, development process, and methodology used in the creation of this project.

## Introduction

The phishing attack continues to be one of the most widespread and harmful types of cyberattack in this current digital environment. It works by tricking people into disclosing their sensitive data like passwords, credit card details, personal identifiers and more by positioning the attackers as trusted organizations. As phishing attacks grow more sophisticated, often using counterfeit websites, misleading emails and even messages generated by AI, users are seriously at risk of being deceived.

Cybersecurity fields have struggled with this issue because traditional spam filters and blacklists are usually more reactive than proactive. Not all phishing attacks come in the form of links or fake websites—some are misleading text messages or social engineering content that can bypass standard detection systems. The rise of AI-created phishing further complicates things, as it allows attackers to generate realistic content at scale.

Without intelligent and real-time detection tools, both individuals and organizations will continue to be exposed to data breaches, financial loss, and reputational damage. Manual

checking is no longer enough, and the growing complexity of phishing attacks means a smarter solution is urgently needed.

## Methodology

### Team Structure and Roles

The development of this phishing detection system was organized and distributed into different roles to ensure efficient and structured progress. Each member was assigned tasks based on technical strengths and project requirements. The roles and their tasks are as follows:

- **Data Engineer**: Responsible for acquiring and preparing the datasets to be used in model training. This included merging datasets from multiple sources, cleaning and preprocessing the data, and generating and saving the TF-IDF vectorizer.

- **Text Model Developer**: Handles the building and training of the core text classification model. This involves selecting appropriate algorithms, handling class imbalances, training the model, and saving the trained model for use in predictions.

- **OCR Developer**: Developed the function that allows users to upload screenshots or images. This includes extracting texts using optical character recognition (OCR) and makes sure extracted texts are properly cleaned to be used in prediction.

- **UI Developer**: Design and develop the web-based user interface. Including building a, carefully thought out, minimalist layout that supports both text and image input and displays model predictions clearly.

- **Evaluation Developer**: Focused on implementing functions for computing performance metrics, generating confidence-based explanations, and identifying misclassifications. This role also included setting up mechanisms for flagging incorrect predictions to improve the model through retraining.

- **Integrator**: Responsible for combining all individual modules (i.e., data preprocessor, OCR, text model, evaluation, and user interface) into a unified application. This included

testing cross-module compatibility, handling errors, and ensuring smooth data flow throughout the system.

Each component was developed in a separate Python script for modularity and ease of testing. The project directory is organized to reflect these roles, and a summary of the file structure is provided (Appendix B). A detailed breakdown of task distribution among team members is provided in (Appendix A).

**Identifying the Problem**

Phishing detection has become increasingly difficult as attackers now use more advanced and deceptive techniques. Many phishing attempts no longer rely on suspicious links or obvious scams. Instead, they use carefully written messages and social engineering tactics that look completely legitimate. These can easily bypass traditional spam filters, blacklists, or rule-based systems that depend on known patterns or URLs.

Current detection methods are mostly reactive and fail to detect zero-day or unseen phishing attacks. Rule-based filters often require constant updates and still miss subtle or cleverly disguised content. Moreover, many users, especially those with little to no knowledge on technologies, rely on manual judgment, which is prone to human error.

Because of this, phishing attacks continue to succeed. There is a clear need for a smarter and more flexible detection system that can analyze both text and images, automatically identify suspicious patterns, and help users who are unaware of prior threats determine whether a message is safe without.

**Objective**

The primary purpose of this project is to create an AI powered phishing detection system that is capable which is capable to analyse user's submitted text or image to identify whether they include features that leads to a phishing attempt, specifically the system aims to:

1. Enhancing cybersecurity by recognizing phishing content before user's engagement

2. Combining machine learning with text and image analysis to spot both obvious and subtle signs of phishing
3. Developing a simple and accessible tool that could help all kinds of people to verify whether a message or image might be a phishing content
4. Supports ongoing learning by incorporating user feedback into the model to enhance future detection accuracy
5. Minimize the human error factor in identifying phishing, which is the primary reason of successful attacks

**Selecting the Approach**

**Literature Review**

Phishing attacks have become increasingly sophisticated as technology advances. Traditional detection systems such as blacklist are not sufficient enough to cope with the evolving growth. According to Gupta et al. (2018), phishing detection techniques can be categorized into:

- **Rule-based method**: This method relies on rules that are manually imputed or regular expressions to detect suspicious behavior, such as known phishing phrases or domains. However, it can't adapt well to new or unknown attacks, and requires constant updates to stay effective.
- **Blacklist-base detection**: These compare URLs, domains, or sender addresses to a blacklist or database that contain known phishing sources. While it is effective against known threats it offers no defense against zero-day phishing (new phishing that hasn't been reported). (Abdelhamid et al., 2014).
- **Machine Learning-based approaches:** It utilizes a machine that can learn from the database it's given. Allowing them to capture the examples of phishing and non-phishing content, which means recognizing new attacks that have not occurred. The feature that it captures could be the frequency of certain words, sentence structure or word pattern. The

examples of train models used are Naive Bayes, SVM (Support Vector Machines), Random Forests, and Deep learning models(Bergholz et al., 2010).

Among these train models, Naive Bayers classifier has proved its strong performance in phishing detection due to its simplicity, interpretability, and low computational cost. There are studies that show Naive Bayers paired well with text-based datasets and are suitable for binary classification problems (Basnet et al., 2012).

In addition to text-based phishing, there are also phishing attempts via images or screenshots. To counter this, some systems have integrated Optical Character Recognition (OCR) to convert those images into text before analysis (Sahingoz et al., 2019). This allows a more in depth analysis and detection even when attackers attempt to hide from filters/detection by using visual content.

**Technology Selection**

Based on a review of existing phishing detection methods and the goals of this project, we selected a combination of technologies that support both text-based and image-based input.

- **Naive Bayes Algorithm**: For detecting phishing in text messages, we used a **Multinomial Naive Bayes classifier** trained on a labeled dataset. This algorithm is widely used for text classification due to its simplicity, fast training, and strong performance with word-frequency features (Basnet et al., 2012).

- **OCR with Tesseract**: To analyze screenshots, we implemented **Tesseract OCR**, an open-source engine developed by Google. It enables the system to extract text from images and pass it to the classifier for phishing analysis (Sahingoz et al., 2019).

- **Python Programming Language**: The system was developed using **Python** due to its strong ecosystem for AI development. We used scikit-learn for machine learning, pytesseract for OCR integration, and tkinter for the user interface.

- **Graphical User Interface (GUI)**: A simple, accessible GUI was developed using **tkinter**. It allows users to input messages or upload screenshots for analysis, making the system easy to use even for non-technical users.

This combination of machine learning, OCR, and a user-friendly interface provides a practical and reliable solution for detecting phishing across different input formats.

**Acquiring and Processing the Data**

To build an accurate AI model for this project, valid datasets are required. Two public datasets, one containing email data and the other SMS messages, were acquired to provide diverse and realistic examples. Once the data has been collected, it must be cleaned and prepared before it can be used to train a machine learning algorithm. Raw messages often included noise, inconsistencies, and irrelevant information that may mislead the model or reduce performance. Since emails and SMS messages differ in structure and tone, we applied slightly different preprocessing techniques to each to maintain their respective characteristics (see *text_helper.py* in Appendix B for implementation details). Data preprocessing plays a vital role in enhancing the quality and effectiveness of the spam/phishing detection system. The following steps were implemented during preprocessing:

1. **Text Normalization**

   Text Normalization ensures that all messages follow a consistent format. The purpose of this step is to simplify the data, reduce vocabulary size, and avoid unnecessary complexity in model training. Steps of text normalization include:

   - Converting all text to lowercase to reduce duplication, as different word cases, e.g. “Phishing”, “phishing”, and “PHISHING”, are treated as three different words unless converted to a common case.

   - Removing characters that are not alphanumeric (e.g., !,?, #, etc). Unless specifically relevant to the task, punctuation and special characters often add noise and increase the vocabulary size without contributing much to the semantic meaning for phishing detection. Removing them simplifies the text and allows the model to focus on the words themselves.

   - Numbers are typically not significant in their meaning for phishing email and texts. Therefore, values can be replaced by a label called “NUM” to simplify processing.

2. **Tokenization**

   Tokenization involves splitting each message into individual units called tokens, typically words. This Tokenization step enables analysis of Individual words, which allows the model to learn from frequency and position.

   Example:

   Original: “Claim your prize now”

   Tokens: [“claim”, “your”, “prize”,”now”]

3. **Stopword Removal**

Stopwords are common words that do not carry significant meaning, such as “the”, “is”,”in”, and “on”. These words were removed using a standard stopword list. The purpose is Stopwords help reducing noise and focuses the model on more meaningful words that are likely to distinguish phishing from legitimate content.

4. **Lemmatization or stemming**

This reduces the number of unique tokens, improving consistency and reducing redundancy in the dataset. This process reduces words to their root form:

- Lemmatization: converts words to dictionary from (e.g., “running” to “run”)
- Stemming: Truncates words to their root (e.g., “messages” to “message”)

5. **Merging Relevant Fields**

In some datasets, phishing indicators may be spread across multiple fields such as subject line, message body and URLs. These were merged into a single text field per message. This allows the model to consider all important information at once rather than analyzing fields in isolation.

6. **Label Encoding**

Machine learning algorithms require numeric labels in order to process and learn from the data. The class labels (i.e.., “Phishing” and “legitimate”) were converted into numerical format:

- 1 for phishing
- 0 for legitimate

After the dataset is cleaned and preprocessed, they are split into training and testing sets using the *train_test_split* function from the scikit-learn library. This is an essential step to ensure that the model can use one portion of the data when trained and tested on a separate, unseen dataset. It helps us evaluate how well the model can generalize to new, real-world data and prevents overfitting.

## Implementation

### User Interface

A user interface is a crucial part of this phishing detection system, which is created to ensure that all the tools are user-friendly, efficient, and easy to navigate for various users. The user interface is developed as a web-based application with a clean and minimalist design, focusing more on usability (see Appendix B for the user interface design). A minimalist design is used to reduce cognitive load, avoid unnecessary distractions, and help users focus directly on the essential tasks such as inputting suspicious content and interpreting results without being overwhelmed by excessive visual elements. This design choice enhances clarity and makes the system more approachable for non-technical users.

The decision to develop the system as a web-based application was made to ensure cross-platform compatibility and ease of access. Users do not need to install any software; they can simply access the system from any device with an internet connection. This makes the tool available to users, supporting practical use in real-time phishing detection scenarios.

The interface consists of these essential sections:

- **Input panel:**
  Users can either enter suspicious texts (e.g. email content or messages) directly into a dedicated input box that supports multi-line inputs and handles content reliably or submit a screenshot that may contain phishing material through a drag-and-drop field or file uploader.

- **Submit button:**
  Once the content is submitted, it will be sent to the AI model for further evaluation. The process runs instantly in the background, and the user receives feedback without needing to reload the page or perform manual preprocessing.

- **Result display:**
  After processing the data, results are displayed clearly along with a brief confidence score and explanation to reassure users on the verdict. The explanation is generated dynamically and simplifies the model's prediction by translating probability values into readable messages that guide users on whether the content is safe or potentially harmful.

**Text Model**

**TF-IDF Vectorizer**

Term Frequency, Inverse-Document-Frequency is a statistical measure that is widely used on NLP (Natural Language Processing). In most cases, the TF-IDF vectorizer is used on text processing machine learning algorithms. It works by observing the whole text data, highlights the words used (e.g more appearances of the same words can reinforce the context of data), or essentially the significance of the word relative to a set of documents, and generates numerical vectors based on that particular observation. The output vectors will then be used as input to classifiers such as Multinomial Naive Bayes, which will be used in this same project.

In the same manner as its naming, TF-IDF is balanced by two logics, TF and IDF. Term Frequency analyses words that appear frequently in the data to draw conclusion that the word is relevant to the topic. Alternately, Inverse Document Frequency is used to balance out the output from TF, since while some words appear so much in the data, they might not mean anything for

example simple words such as “or”, “while”. Since this algorithm works with data frequency, probability is the core logic here.

This vectorizer is relevant and effective for a phishing email detector, since it can match words that are normally found (template) on many phishing emails, thus drawing a valid conclusion (TF does this). Subsequently, the IDF can filter out irrelevant words so that a valid conclusion can still be preserved.

**Multinomial Naive Bayes**

Multinomial Naive Bayes is a classifier that is used in this project. This classifier works similarly with TF-IDF, it continues TF-IDF’s work, plus both of them work with probabilistic theories.

The foundation of MultinomialNB is Bayes’ theorem common in probabilistic analysis. It is further extended to be naive, that each word present does not affect the conclusion that can be drawn from another word, meaning that all features are independent of each other given the class label**.** (Sriram, 2025) It also adopts the feature of a multinomial distribution, where the word counts for a fixed number of trials are emphasized as well as Laplace Smoothing such that features of new samples can be handled in a correct manner (i.e, does not directly affect the overall output). For example, appearances of a new word in a new data sample must be familiarized with the algorithm so that it can correctly label them as spam or ham, by inserting more data.

To be precise, the naive bayes formula utilized is as follows:

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

Here, P(A|B) refers to the posterior probability (the probability that hypothesis A is true in presence of the event B). P(A) and P(B) indicate the probability of hypothesis A occurring

without any external factor and probability of evidence, respectively. Finally, P(B|A) represents the probability of evidence while hypothesis A is true. By utilizing all this information, we can compute the posterior probability after data is converted to frequency distribution and likelihood table.

In general, the multinomial naive bayes algorithm draws conclusions on the category of each data, in this case whether a message is considered as a phishing or legitimate message, based on the number of occurrences of a relevant word.

Both of these algorithms can be imported directly from *sklearn*'s library when working with Python. Therefore, it is not required to manually implement them.

In the process of training the model to be used for classifying the emails or sms, the data samples are loaded and further processed by the TF-IDF vectorizer. The output, numerical vectors are later inputted to MultinomialNB classifier, thus generating a model that can be saved onto a joblib extension file (handles machine learning models). These can then be used for labelling new datasets, spam or ham with the existing model and vectorizer as its parameters.

**OCR Module**

An OCR (Optical Character Recognition) was implemented to allow users to analyze phishing messages from images and screenshots. The OCR extracts texts from the uploaded images to enable the AI phishing detection model to evaluate the messages provided, even if they are attached in the form of pictures and not raw text format. This feature enhances the AI models' usability for identifying phishing attempts in the form of visuals such as email snapshots, chat screenshots, or mobile message images.

To implement this functionality, Tesseract OCR was selected. Which is an open-source OCR engine developed by Google and widely used for its:

- Accurate and reliable for screen-based or printed text
- Support multiple languages for broad application.
- Easy integration with Python using the pytesseract wrapper.

Using pytesseract, we integrated Tesseract into our Python-based system, and the image preprocessing was handled using the Pillow (PIL) library. The tesseract was installed locally on the operating system, and we used direct path referencing for accessing and loading pictures.

Key tools used:

- Tesseract OCR: The main engine that converts images into text.
- pytesseract: Python wrappers that connect Tesseract to Python scripts.
- Pillow (PIL): Library for image loading and preprocessing, which helps open and edit images.

The OCR module is responsible for handling the uploaded image files, and then preparing them for OCR, and extracting any readable text. Below is an overview of the implementation workflow:

1. Image Upload: The user uploads a screenshot through the system interface.
2. Preprocessing: The image is cleaned and adjusted to improve clarity for OCR.
3. Text Extraction: Tesseract processes the image and outputs the detected text.
4. Integration: The text is passed to the phishing detection model for analysis.

This process is seamless, so that users can interact with the system using either typed text or visual inputs.

To improve OCR's accuracy, especially for low-resolution or blurry screenshots, we applied some preprocessing techniques before passing them to Tesseract. These methods were selected to optimize our pictures:

- **Grayscale Conversion**: Removes color from images, to help OCR focus on the text
- **Upscaling**: Enlarges the image to double the size of the original. It improves the reading of fonts as it has become more readable and character definition.

- **Sharpening**: Enhances edges and character outlines. It prevents OCR from misreading blurry or soft edges.

These enhancements improve the OCR output, particularly for screenshots captured on mobile devices or compressed image formats.

*Code snippet:*

```
# Image Preprocessing

# Convert to grayscale

image = image.convert("L")



# Resize (upscale by 2x to help OCR if image is small)

image = image.resize((image.width * 2, image.height * 2))



# Sharpen the image

image = image.filter(ImageFilter.SHARPEN)
```

Once the text is extracted from the screenshot, it is passed into the phishing detection module, the same as the text input flow. This action is handled by the integrator, who ensures the smooth relationship between the OCR, the AI model, and the interface. The result (Phishing / Not Phishing), along with a confidence score, is then shown to the user via the interface.

**Integration**

The integration phase involves combining the various components of the phishing detection system into a cohesive application. This integration allows the system to seamlessly handle inputs, data preprocessing, make predictions, logging, and model retraining. Key aspects of the integration include:

1. User Interaction and Input Handling

The system provides a user interface (UI) that allows users to input text or upload images. When a user uploads an image or enters text, the UI captures this input and passes it to the *process_image_and_predict* function, returning the prediction verdict (phishing or not) and confidence score for the user to see. This function serves as the central processing point, handling both text and image inputs.

2. Processing Input Data

   The system processes input data, which can either be text or an image. If an image is provided, the OCR module extracts text from the image using the *extract_text_from_image* function from the *OCR* module. The input text is then vectorized using the *vectorizer.transform* method from the loaded vectorizer to ensure that the input data is in the correct format for the machine learning to process.

3. Prediction

   Within the *process_image_and_predict* function, the preprocessed text is passed through the trained machine learning model in which new data can be inserted into a method available. This will return true/false representations of spam/ham in the specified dataset. After prediction is made, the *evaluate_model* function from the *evaluation* module will be triggered, evaluating the model's evaluation metrics and displaying them in the terminal/command prompt.

4. Logging

   The results of each prediction, including the input text, prediction label, confidence score, and timestamp,are logged into a csv file using the *log_prediction* function. This logging step is done to ensure that all predictions are recorded for future analysis model retraining.

5. Retraining

   The machine learning model is updated with new data periodically to maintain its accuracy. Predictions in the log file are reviewed by developers and results with inaccurate predictions will be flagged and marked with the correct label. The prediction

log file is loaded, during retraining, by the *retrain_from_flagged* function which will filter out entries with valid corrections and then retrain the model.

**Evaluation**

To find out how well the phishing detection model identified messages as either authentic or phishing, an evaluation procedure was conducted. A different test dataset was employed for this. These messages are appropriate for evaluating the overall performance of the model because they were manually tagged and excluded during training.

The same TF-IDF vectorizer used for training was utilized to clean and convert the test data into numerical representation prior to evaluation. As a result, the model was able to process messages consistently. These TF-IDF vectors were then used to inform predictions made by the trained model.

Several standard evaluation metrics were computed using scikit-learn's built-in functions to determine the model's performance:

- **Accuracy**: The percentage of accurate forecasts among all forecasts.
- **Precision**: How many of the mails that were identified as phishing were, in fact, phishing.
- **Recall**: How many phishing messages were accurately identified out of all the real ones.
- **F1 Score**: A more comprehensive picture of performance by striking a balance between recall and precision.

A classification threshold of 0.6 was applied. This means the model only flags a message as phishing if it is at least 60% confident. This helps reduce false positives by requiring a higher level of certainty before showing a phishing warning.

For each prediction, the *predict_proba* function was used to generate a confidence score, indicating how sure the model was. A custom function called *explain_verdict* was used to turn this into a clear message, such as:

- "The model is 91.00% confident that this message is a phishing message. ❗ Do not interact ❗ ", or

- "The model is 93.00% confident that this message is not a phishing message.″

To check whether the model made any mistakes, another function called *identify_misclassifications* was used to compare the predictions on test data with the actual labels. This step is important because even if the model shows high accuracy overall, a few incorrect predictions, especially false negatives where phishing messages are labeled as safe, can be dangerous. Identifying misclassifications helps us spot those specific cases and better understand where the model might fail in real-world use.

## Validation and Verification

Validation assesses whether the system achieves its intended goal which is to help users identify phishing messages accurately and reliably. We tested these by conducting:

- **Functional Testing**: The system was used to process both typed messages and screenshots through the actual user interface. The output included a phishing verdict along with a confidence score, which matched expectations based on the content submitted.

- **Usability Testing**: The interface was tested to make sure that the prediction verdicts were clearly worded and understandable. This helped confirm that the system can be used even by non-technical users.

- **Logging and Feedback Loop**: Predictions are automatically logged with the message content, prediction, confidence, and timestamp. These logs can be reviewed to trace outcomes, and flagged mistakes can be used to retrain the model. This contributes to long-term validation by ensuring the system can continue improving and adjusting to user feedback.

Verification focuses on confirming that each part of the system was implemented correctly and worked as expected from a technical standpoint. This included:

- **Unit Testing:** Individual tests were conducted for key functions like process_image_and_predict, log_prediction, identify_misclassifications, and retrain_from_flagged. With various sample inputs to check their outputs and confirm that they handled edge cases (such as empty input or unexpected data) without crashing.

- **Internal Checks**: Although we did not automate file integrity tests, we verified during development that key resources like the model and vectorizer loaded correctly using exception handling. We also ensured input data was cleaned and vectorized properly, which prevented runtime crashes and confirmed the system could handle typical input issues.

- **Integration Testing**: Different modules, including data preprocessing, OCR for image inputs, and model prediction, were tested together to make sure they operated smoothly as a connected system. This avoided problems like misaligned data formats or broken function calls and helped guarantee consistency between components.

## Result and Analysis

The results of the phishing detection system, including model performance, validation initiatives, and system level verification, are shown in this section. The results demonstrate that the system successfully meets its intended goal: to assist users in identifying phishing messages reliably and accurately, whether the input is in text form or as an image.

### Data Collection and Preprocessing

Two publicly available datasets were used:

- SpamAssassin Email Dataset: Contains a collection of labeled email messages for spam and ham classification.

- UCI SMS Spam Collection: Comprises SMS messages labeled as either spam or legitimate (ham).

After cleaning, removing duplicates, and standardizing formats, a total of 11,146 clean samples were retained. These were preprocessed using TF-IDF vectorization, with a feature limit of 5,000, to convert text into numerical form suitable for model input.

The dataset was split into training and testing sets using an 80/20 ratio, stratified to maintain class balance:

- **Training set**: 8,916 samples
- **Test set**: 2,230 samples

To address class imbalance during training, oversampling techniques such as SMOTE were applied to ensure the model was not biased toward the majority class.

**Model Training and Evaluation**

A Multinomial Naive Bayes classifier was used to train the model on vectorized text. The model was then evaluated on the test set using the following classification metrics (see Appendix C for full evaluation metrics):

- **Accuracy**: 0.95
- **Precision**: 0.88
- **Recall**: 0.91
- **F1-score**: 0.89

These results indicate strong performance. The high recall confirms the system's ability to correctly identify phishing messages, while the high precision shows it avoids labeling too many legitimate messages as phishing. The F1-score reflects a good balance between the two.

**Observations and Limitations**

- Our model handles clear phishing messages well but can struggle with ambiguous or highly sophisticated social engineering content. However, better outcomes might be achieved after model retraining.

- The OCR pipeline depends on the quality of the image. Slight text loss or formatting changes can reduce accuracy.

- During development, certain environment-specific issues such as Tesseract path resolution were observed. To address this, a fallback strategy using *shutil.which("tesseract")* was implemented to dynamically locate the executable. However, this approach may not work on all systems depending on how Tesseract is installed. In such cases, hardcoding the full path to the Tesseract executable is a known solution. Although this alternative was tested during development, it was not included in the submitted version to avoid platform-specific dependencies.

## Conclusion

This project provides the outcome of a phishing email detector that accepts inputs in the form of text and image. It has a graphical user interface built with python, and overall functionalities featuring different libraries that are available to the python language for example scikitlearn, pytesseract, nltk, etc. Moreover, the software implements Natural Language Processing as the basis of the whole project, with the tasks carried out by TF-IDF vectorizer and Multinomial Naive Bayes classifier. To enable the software to process images, OCR capabilities are also added.

After training and testing, it is concluded that the software reached an accuracy of **95%**, thus implying the suitability and relatability of the different types of mechanism used as well as its reliability to detect malicious messages. Although proper, in the future, this project can be further improved by pairing with advanced deep learning models, or with new technology as it emerges to replace the current workflow.

## References


GeeksforGeeks. (2025, February 7). *Understanding TFIDF (Term FrequencyInverse Document Frequency)*. GeeksforGeeks. https://www.geeksforgeeks.org/understanding-tf-idf-term-frequency-inverse-document-frequency/

Jain, A. (2024, February 4). TF-IDF in NLP (Term Frequency Inverse Document Frequency). *Medium*. https://medium.com/@abhishekjainindore24/tf-idf-in-nlp-term-frequency-inverse-document-frequency-e05b65932f1d

Sriram. (2025, May 27). *Multinomial naive Bayes explained: Function, advantages & disadvantages, applications*. upGrad Blog. https://www.upgrad.com/blog/multinomial-naive-bayes-explained/

GeeksforGeeks. (2025b, May 30). *Multinomial naive Bayes*. GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/multinomial-naive-bayes/

## Appendices

### Appendix A: Team Members Task Distribution

This table shows a detailed breakdown of the task distribution among members.

| Role | Team Member | Task |
|---|---|---|
| Data Engineer | Mok Isabella (CYS2309274) | Acquired, cleaned, and preprocessed datasets; created and saved the TF-IDF vectorizer. |
| Text Model Developer | Dillon Nicholas (CYS2309271) | Built and trained the phishing detection model; handled vectorized input and prediction logic; performed system debugging. |
| OCR Developer | Stefanie (CYS2309276) | Implemented text extraction from image uploads using OCR; handled image-to-text conversion. |
| Evaluation Developer | Willim Raymond Russel (CYS2309270) | Designed and implemented the user interface; ensured user-friendly layout and functionality. |
| User Interface Developer | Christian Kenny (CYS2309277) | Implemented performance metric evaluations, misclassification identification, and confidence score generation. |

| | | |
|---|---|---|
| Integrator | Itanza Joan (CYS2309273) | Integrated all modules (data, model, OCR, evaluation, UI); performed system-level debugging and final testing. |

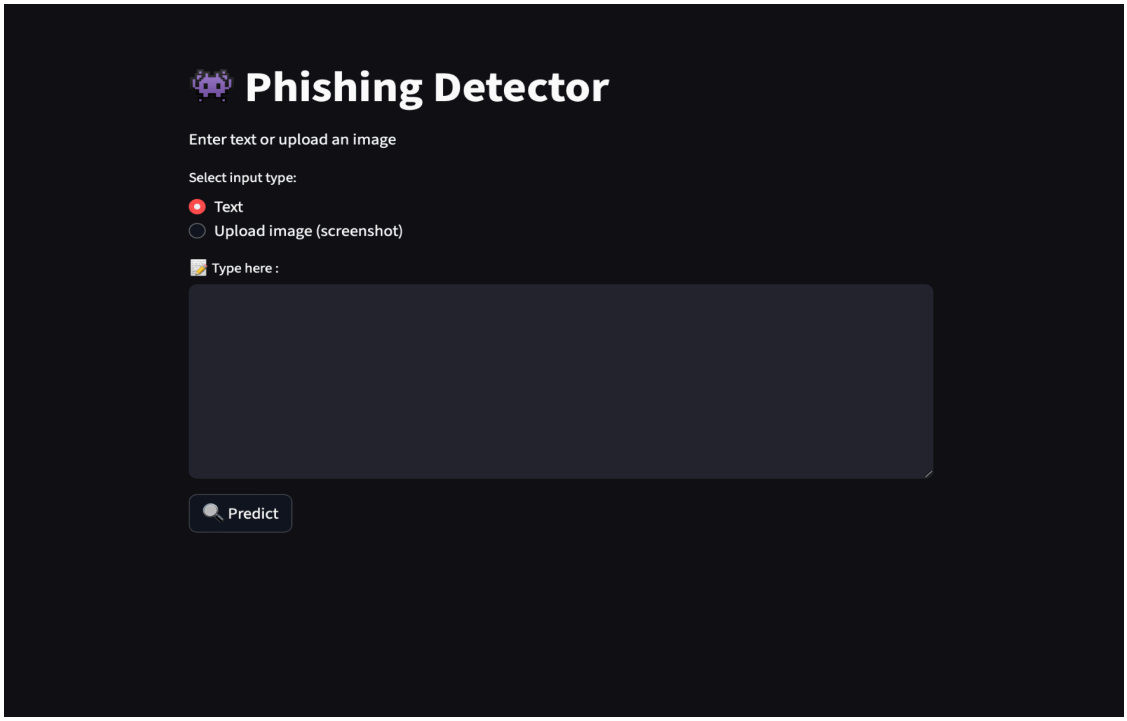
**Appendix B: Directory and Files**

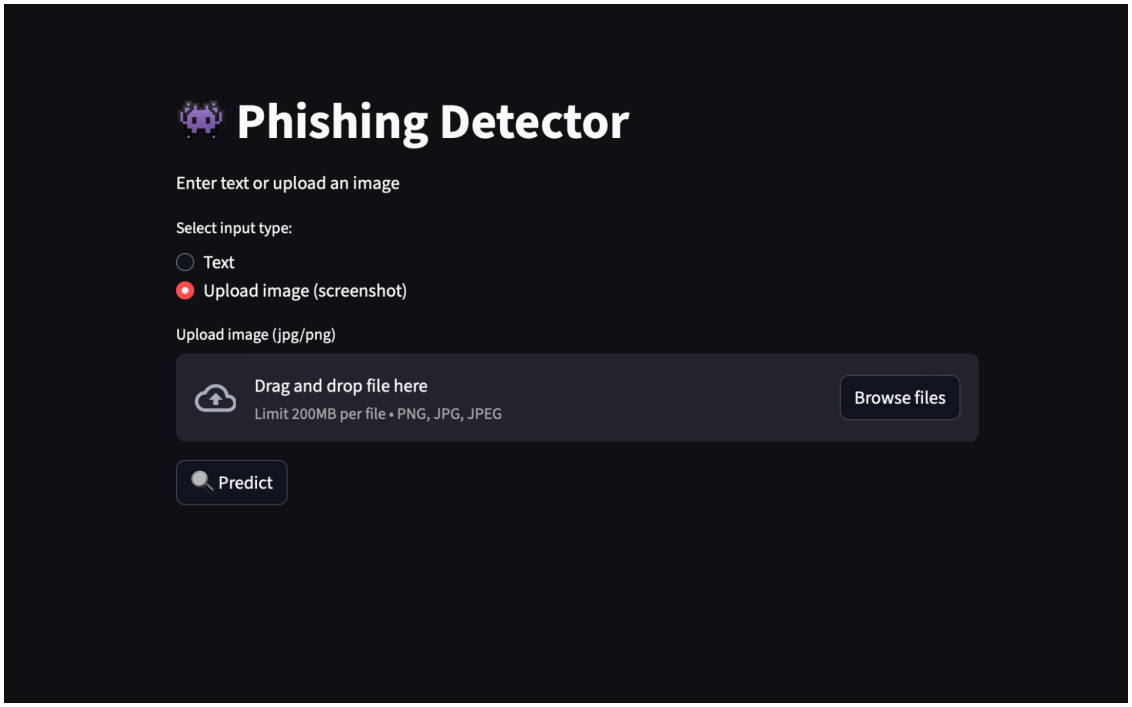
The full source codes, processed datasets, and output logs are stored in a structured directory. Due to file size, the directory can be accessed via the link below.

https://drive.google.com/drive/folders/1QxyGWXNYKDQGCPuU9hdIrZC9HcSzDD-e?usp=sharing

**Appendix C: Graphic User Interface Design**

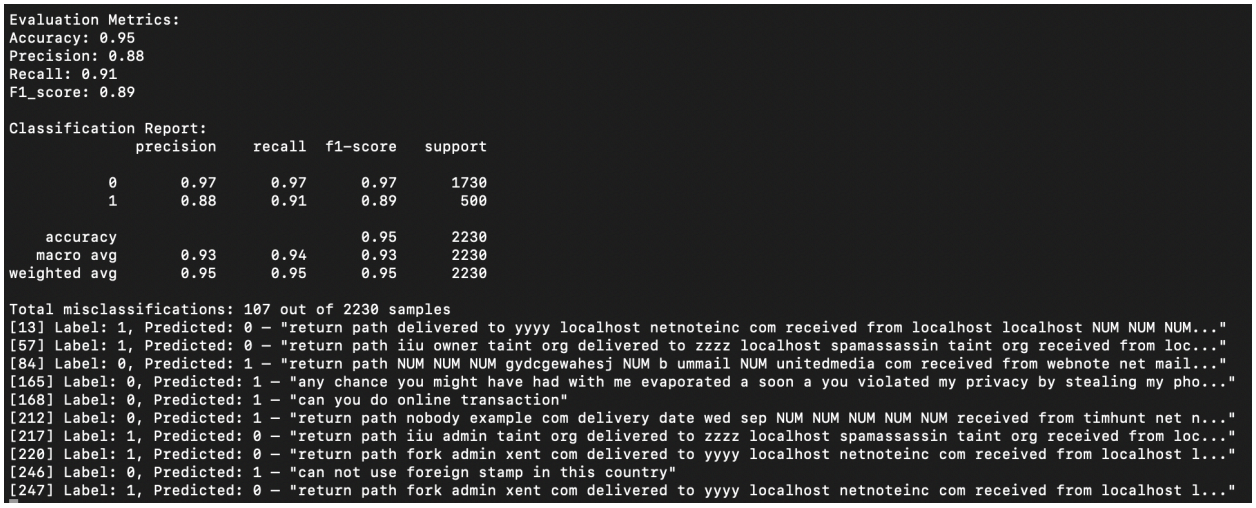
This figure shows what users will see when they run our user interface (top: text input, bottom: image input).



## Appendix D: Model Evaluation Metrics

The figure shows a screenshot of the evaluation metrics for our model.



```
Evaluation Metrics:
Accuracy: 0.95
Precision: 0.88
Recall: 0.91
F1_score: 0.89

Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.97      0.97      1730
           1       0.88      0.91      0.89       500

    accuracy                           0.95      2230
   macro avg       0.93      0.94      0.93      2230
weighted avg       0.95      0.95      0.95      2230

Total misclassifications: 107 out of 2230 samples
[13] Label: 1, Predicted: 0 — "return path delivered to yyyy localhost netnoteinc com received from localhost localhost NUM NUM NUM..."
[57] Label: 1, Predicted: 0 — "return path iiu owner taint org delivered to zzzz localhost spamassassin taint org received from loc..."
[84] Label: 0, Predicted: 1 — "return path NUM NUM NUM gydcgewahesj NUM b ummail NUM unitedmedia com received from webnote net mail..."
[165] Label: 0, Predicted: 1 — "any chance you might have had with me evaporated a soon a you violated my privacy by stealing my pho..."
[168] Label: 0, Predicted: 1 — "can you do online transaction"
[212] Label: 0, Predicted: 1 — "return path nobody example com delivery date wed sep NUM NUM NUM NUM NUM received from timhunt net n..."
[217] Label: 1, Predicted: 0 — "return path iiu admin taint org delivered to zzzz localhost spamassassin taint org received from loc..."
[220] Label: 1, Predicted: 0 — "return path fork admin xent com delivered to yyyy localhost netnoteinc com received from localhost l..."
[246] Label: 0, Predicted: 1 — "can not use foreign stamp in this country"
[247] Label: 1, Predicted: 0 — "return path fork admin xent com delivered to yyyy localhost netnoteinc com received from localhost l..."
```

## APPENDIX 1

## MARKING RUBRICS

| | | Score and Descriptors | | | | |
|---|---|---|---|---|---|---|
| **No.** | | **Poor** | **Average** | **Excellent** | **Weight (%)** | **Mark** |
| **Component 1: Project Development** | | | | | | |
| | | **0 - 4** | **5 - 7** | **8 - 10** | **10** | |
| 1 | Code quality | Codes are poorly structured.<br>Not fully functional.<br>Not documented. | Codes are sufficiently documented and mostly functional.<br>Satisfactorily structured | Codes are fully functional and well structured and documented | | |
| | | **0 - 4** | **5 - 7** | **8 - 10** | **10** | |
| 2 | Implementation of algoritm related to AI | Incomplete development of the method. | Complete development of method. | Completed\ enhanced the developed methods. | | |
| | | **0-2** | **3** | **4-5** | **5** | |
| 3 | Completeness and complexity of your project | The functionalities that are implemented are non-functional | All functionalities are implemented and functional | All the functionalities are fully implemented and functional<br>Implemented extra functionalities. | | |
| | | | | **Subtotal** | **25** | |
| **Component 2: Project Report + Project Demonstration** | | | | | | |
| | | **0 - 4** | **5 - 7** | **8 - 10** | **10** | |
| 1 | Project Report | Poor writing quality<br>Poor or no formatting / presentation<br>Lack of technical content | Satisfactory writing quality, grammar and flow.<br>Substantial technical content with used method. | Good writing quality, grammar and flow<br>Well formatted and good presentation<br>Demonstrate excellent technical content based the method used | | |
| | | **0 - 4** | **5 - 7** | **8 - 10** | **10** | |
| 2 | Presentation | Poor quality slides<br>Poor time management<br>Speech that is unclear | Satisfactory quality slides<br>Speech that is satisfactory and understandable | High quality slides<br>Good time management<br>Speech that is clear and impactful | | |
| | | **0-2** | **3** | **4-5** | **5** | |
| 3 | Demonstration | Poor demonstration that is unclear<br>The developed application/research is not functioning/implemented fully | Satisfactory demonstration<br>The developed application/research is partially functioning/implemented fully | Good demonstration<br>The developed application/research is fully functioning and logical | | |
| | | | | **Subtotal** | **25** | |
| | | | | **Grand Total** | **50** | |

Note to students: Please attach this appendix together with the submission of coursework

# Project _SOF106 2504_Group 9.pdf

ORIGINALITY REPORT

| 5% | 3% | 3% | 2% |
|---|---|---|---|
| SIMILARITY INDEX | INTERNET SOURCES | PUBLICATIONS | STUDENT PAPERS |

PRIMARY SOURCES

| | | |
|---|---|---|
| 1 | R. N. V. Jagan Mohan, B. H. V. S. Rama Krishnam Raju, V. Chandra Sekhar, T. V. K. P. Prasad. "Algorithms in Advanced Artificial Intelligence - Proceedings of International Conference on Algorithms in Advanced Artificial Intelligence (ICAAAI-2024)", CRC Press, 2025<br>Publication | 1% |
| 2 | www.mdpi.com<br>Internet Source | <1% |
| 3 | www.upgrad.com<br>Internet Source | <1% |
| 4 | core.ac.uk<br>Internet Source | <1% |
| 5 | Submitted to De Montfort University<br>Student Paper | <1% |
| 6 | www.coursehero.com<br>Internet Source | <1% |
| 7 | Submitted to Queen's College<br>Student Paper | <1% |
| 8 | iris.polito.it<br>Internet Source | <1% |
| 9 | Sherif Elmitwalli, John Mehegan. "Sentiment analysis of COP9-related tweets: a comparative study of pre-trained models and traditional techniques", Frontiers in Big Data, 2024<br>Publication | <1% |

| | | |
|---|---|---|
| 10 | Submitted to University of Glasgow<br>Student Paper | <1% |
| 11 | dokumen.pub<br>Internet Source | <1% |
| 12 | Submitted to Manchester Metropolitan University<br>Student Paper | <1% |
| 13 | Submitted to Taylor's Education Group<br>Student Paper | <1% |
| 14 | ijarsct.co.in<br>Internet Source | <1% |
| 15 | isg-konf.com<br>Internet Source | <1% |
| 16 | docs.microsoft.com<br>Internet Source | <1% |
| 17 | ebin.pub<br>Internet Source | <1% |
| 18 | eitca.org<br>Internet Source | <1% |
| 19 | irjet.net<br>Internet Source | <1% |

| | | | |
|---|---|---|---|
| Exclude quotes | On | Exclude matches | Off |
| Exclude bibliography | On | | |